Lab 7: C Modeling and Sine Wave Generator

ECEN 127L: Tuesday 5:15 - 8:00 pm

Ryan Kiniris and Dylan Thornburg

Introduction

In this lab, we are making, implementing, and verifying a sine-wave generator using two different types of hardware architectures that mirror the behavior of the given C model. The goal is to design a Verilog system that accurately symbolizes the multi-stage datapath of the waveform generator, including systems like phase accumulator, wavetable memory, fractional interpolation, and rounding stages. Both architectures must replicate the precise timing and arithmetic operations described in the lab handout and must exactly match the modified C reference model.

The primary focus is to gain practical experience with hardware-accurate modeling, pipelined datapath design, fixed-point arithmetic, and waveform interpolation techniques. By modifying the C model, creating corresponding Verilog modules, and validating cycle-by-cycle equivalence through a testbench, we establish a deeper understanding of architecture verification and formal model alignment.

Procedure

We had started off by updating the C model to match the lab architecture, adjusting the phase format, wavetable size, and sample-generation logic. After confirming that C output is correct, we built the Verilog version of the sine generator, implementing the accumulator, ROM lookup, interpolation, and rounding modules, then combining them into a top-level design.

A testbench was created to generate two sine-wave cycles and compare each Verilog output to the C results. For Part 2, we changed the sample rate and repeated the verification. Finally for Part 3, we implemented the alternate architecture that outputs samples every two cycles, updated the verilog architecture accordingly, and verified both 80- and 100-sample-per-cycle outputs for correctness. We know all architectures are correct because they match the C code output and their respective logic was derived from the C model itself.

Results

Part 1:

Diff (should output no numbers if all match):

```
[dthornburg@linux402604 part1]$ diff  <(./simv -no_save) <(./main)
1,3d0
< Chronologic VCS simulator copyright 1991-2023
< Contains Synopsys proprietary information.
< Compiler version V-2023.12-SP1-1_Full64; Runtime version V-2023.12-SP1-1_Full6
4;  Nov 23 17:38 2025
404,409d400
< $finish called from file "tb.v", line 46.
< $finish at simulation time                 1208
<           V C S   S i m u l a t i o n   R e p o r t
< Time: 1208
< CPU Time:      0.180 seconds;       Data structure size:   0.0Mb
< Sun Nov 23 17:38:50 2025
[dthornburg@linux402604 part1]$
```

Sdiff (output all numbers when matching):

```
[dthornburg@linux402604 part1]$ sdiff  <(./simv -no_save) <(./main)
Chronologic VCS simulator copyright 1991-2023                <
Contains Synopsys proprietary information.                   <
Compiler version V-2023.12-SP1-1_Full64; Runtime version V-20 <
2032                                                            2032

2170                                                            2170

2293                                                            2293

2416                                                            2416

2555                                                            2555

2678                                                            2678

2790                                                            2790

2907                                                            2907

3026                                                            3026

3137                                                            3137

3247                                                            3247

3348                                                            3348

3436                                                            3436
```

3528	3528
3610	3610
3692	3692
3766	3766
3827	3827
3889	3889
3945	3945
3986	3986
4027	4027
4050	4050
4070	4070
4080	4080
4080	4080
4080	4080
4071	4071
4050	4050
4028	4028
3987	3987
3946	3946
3891	3891
3829	3829
3768	3768
3694	3694
3612	3612
3530	3530
3438	3438
3350	3350

3250	3250
3140	3140
3029	3029
2911	2911
2792	2792
2681	2681
2558	2558
2419	2419
2297	2297
2174	2174
2035	2035
1912	1912
1789	1789
1666	1666
1528	1528
1405	1405
1292	1292
1175	1175
1056	1056
0946	0946
0835	0835
0733	0733
0646	0646
0553	0553
0472	0472
0390	0390
0315	0315
0253	0253

0192	0192
0135	0135
0094	0094
0053	0053
0030	0030
0009	0009
0000	0000
0000	0000
0000	0000
0008	0008
0028	0028
0050	0050
0091	0091
0131	0131
0187	0187
0248	0248
0310	0310
0383	0383
0465	0465
0547	0547
0638	0638
0726	0726
0826	0826
0935	0935
1047	1047
1165	1165
1284	1284

1395	1395
1518	1518
1656	1656
1779	1779
1902	1902
2025	2025
2164	2164
2286	2286
2409	2409
2548	2548
2671	2671
2784	2784
2900	2900
3021	3021
3130	3130
3241	3241
3344	3344
3430	3430
3523	3523
3605	3605
3687	3687
3763	3763
3824	3824
3885	3885
3943	3943
3984	3984
4025	4025
4049	4049

4069	4069
4080	4080
4080	4080
4080	4080
4072	4072
4052	4052
4031	4031
3990	3990
3949	3949
3894	3894
3832	3832
3771	3771
3698	3698
3617	3617
3535	3535
3444	3444
3355	3355
3255	3255
3147	3147
3035	3035
2917	2917
2798	2798
2688	2688
2565	2565
2426	2426
2303	2303
2180	2180

2043	2043
1919	1919
1796	1796
1673	1673
1534	1534
1412	1412
1298	1298
1182	1182
1061	1061
0952	0952
0841	0841
0738	0738
0652	0652
0558	0558
0476	0476
0394	0394
0318	0318
0257	0257
0195	0195
0137	0137
0096	0096
0055	0055
0031	0031
0010	0010
0000	0000
0000	0000
0000	0000
0006	0006

```
0027                                                    0027
0047                                                    0047
0088                                                    0088
0129                                                    0129
0183                                                    0183
0245                                                    0245
0306                                                    0306
0378                                                    0378
0460                                                    0460
0542                                                    0542
0632                                                    0632
0722                                                    0722
0821                                                    0821
0929                                                    0929
1042                                                    1042
1158                                                    1158
1278                                                    1278
1388                                                    1388
1511                                                    1511
1650                                                    1650
1772                                                    1772
1895                                                    1895

$finish called from file "tb.v", line 46.                <
$finish at simulation time          1208                <
         V C S   S i m u l a t i o n   R e p o r t      <
Time: 1208                                               <
CPU Time:      0.170 seconds;       Data structure size:   0. <
Sun Nov 23 17:40:51 2025                                 <
[dthornburg@linux402604 part1]$
```

Part 2:

Diff (should output no numbers if all match):

```
[dthornburg@linux402604 part2]$ diff  <(./simv -no_save) <(./main)
1,3d0
< Chronologic VCS simulator copyright 1991-2023
< Contains Synopsys proprietary information.
< Compiler version V-2023.12-SP1-1_Full64; Runtime version V-2023.12-SP1-1_Full6
4;  Nov 23 17:29 2025
304,309d300
< $finish called from file "tb.v", line 46.
< $finish at simulation time                 908
<           V C S   S i m u l a t i o n   R e p o r t
< Time: 908
< CPU Time:      0.180 seconds;       Data structure size:   0.0Mb
< Sun Nov 23 17:29:36 2025
[dthornburg@linux402604 part2]$
```

Sdiff (output all numbers when matching):

```
[dthornburg@linux402604 part2]$ sdiff  <(./simv -no_save) <(./main)
Chronologic VCS simulator copyright 1991-2023                <
Contains Synopsys proprietary information.                   <
Compiler version V-2023.12-SP1-1_Full64; Runtime version V-20 <
2032                                                             2032

2211                                                             2211

2375                                                             2375

2555                                                             2555

2718                                                             2718

2866                                                             2866

3026                                                             3026

3177                                                             3177

3315                                                             3315

3435                                                             3435

3555                                                             3555

3664                                                             3664

3766                                                             3766

3847                                                             3847
```

3929	3929
3986	3986
4036	4036
4063	4063
4080	4080
4080	4080
4078	4078
4051	4051
4015	4015
3961	3961
3891	3891
3809	3809
3722	3722
3613	3613
3504	3504
3379	3379
3251	3251
3101	3101
2954	2954
2794	2794
2642	2642
2468	2468
2299	2299
2134	2134
1955	1955
1792	1792
1625	1625

1448	1448
1294	1294
1137	1137
0989	0989
0837	0837
0708	0708
0583	0583
0474	0474
0364	0364
0275	0275
0193	0193
0122	0122
0068	0068
0030	0030
0003	0003
0000	0000
0000	0000
0014	0014
0041	0041
0089	0089
0144	0144
0226	0226
0308	0308
0408	0408
0517	0517
0634	0634
0755	0755
0891	0891

1044	1044
1202	1202
1349	1349
1513	1513
1693	1693
1856	1856
2020	2020
2200	2200
2363	2363
2543	2543
2707	2707
2855	2855
3016	3016
3166	3166
3305	3305
3426	3426
3547	3547
3656	3656
3760	3760
3842	3842
3924	3924
3982	3982
4034	4034
4061	4061
4080	4080
4080	4080
4080	4080

4053	4053
4019	4019
3964	3964
3897	3897
3815	3815
3730	3730
3621	3621
3512	3512
3387	3387
3261	3261
3113	3113
2965	2965
2804	2804
2654	2654
2481	2481
2310	2310
2147	2147
1967	1967
1803	1803
1638	1638
1460	1460
1304	1304
1148	1148
0999	0999
0847	0847
0715	0715
0590	0590
0481	0481

```
0372                                                    0372
0281                                                    0281
0199                                                    0199
0126                                                    0126
0072                                                    0072
0032                                                    0032
0005                                                    0005
0000                                                    0000
0000                                                    0000
0012                                                    0012
0039                                                    0039
0085                                                    0085
0140                                                    0140
0220                                                    0220
0302                                                    0302
0400                                                    0400
0509                                                    0509
0625                                                    0625
0745                                                    0745
0882                                                    0882
1034                                                    1034
1190                                                    1190
1339                                                    1339
1502                                                    1502
1681                                                    1681
1845                                                    1845
$finish called from file "tb.v", line 46.          <
$finish at simulation time          908            <
```

```
      V C S   S i m u l a t i o n   R e p o r t        <
Time: 908                                                <
CPU Time:      0.180 seconds;       Data structure size:   0. <
Sun Nov 23 17:31:48 2025                                 <
[dthornburg@linux402604 part2]$
```

Part 3 100 Samples:

Diff (should output no numbers if all match):

```
[dthornburg@linux402604 samples_100]$ diff  <(./simv -no_save | sed '1,3d') <(./
main)
401,406d400
< $finish called from file "tb.v", line 40.
< $finish at simulation time                 1608
<           V C S   S i m u l a t i o n   R e p o r t
< Time: 1608
< CPU Time:      0.180 seconds;       Data structure size:   0.0Mb
< Mon Nov 24 18:27:16 2025
[dthornburg@linux402604 samples_100]$
```

Sdiff (output all numbers when matching):

```
[dthornburg@linux402604 samples_100]$ sdiff  <(./simv -no_save | sed '1,3d') <(./main)
2032                                                     2032

2170                                                     2170

2293                                                     2293

2416                                                     2416

2555                                                     2555

2678                                                     2678

2790                                                     2790

2907                                                     2907

3026                                                     3026

3137                                                     3137

3247                                                     3247
```

3348	3348
3436	3436
3528	3528
3610	3610
3692	3692
3766	3766
3827	3827
3889	3889
3945	3945
3986	3986
4027	4027
4050	4050
4070	4070
4080	4080
4080	4080
4080	4080
4071	4071
4050	4050
4028	4028
3987	3987
3946	3946
3891	3891
3829	3829
3768	3768
3694	3694
3612	3612
3530	3530
3438	3438

3350

3250

3140

3029

2911

2792

2681

2558

2419

2297

2174

2035

1912

1789

1666

1528

1405

1292

1175

1056

0946

0835

0733

0646

0553

0472

0390

3350

3250

3140

3029

2911

2792

2681

2558

2419

2297

2174

2035

1912

1789

1666

1528

1405

1292

1175

1056

0946

0835

0733

0646

0553

0472

0390

0315	0315
0253	0253
0192	0192
0135	0135
0094	0094
0053	0053
0030	0030
0009	0009
0000	0000
0000	0000
0000	0000
0008	0008
0028	0028
0050	0050
0091	0091
0131	0131
0187	0187
0248	0248
0310	0310
0383	0383
0465	0465
0547	0547
0638	0638
0726	0726
0826	0826
0935	0935
1047	1047
1165	1165

1284	1284
1395	1395
1518	1518
1656	1656
1779	1779
1902	1902
2025	2025
2164	2164
2286	2286
2409	2409
2548	2548
2671	2671
2784	2784
2900	2900
3021	3021
3130	3130
3241	3241
3344	3344
3430	3430
3523	3523
3605	3605
3687	3687
3763	3763
3824	3824
3885	3885
3943	3943
3984	3984

4025	4025
4049	4049
4069	4069
4080	4080
4080	4080
4080	4080
4072	4072
4052	4052
4031	4031
3990	3990
3949	3949
3894	3894
3832	3832
3771	3771
3698	3698
3617	3617
3535	3535
3444	3444
3355	3355
3255	3255
3147	3147
3035	3035
2917	2917
2798	2798
2688	2688
2565	2565
2426	2426
2303	2303

2180	2180
2043	2043
1919	1919
1796	1796
1673	1673
1534	1534
1412	1412
1298	1298
1182	1182
1061	1061
0952	0952
0841	0841
0738	0738
0652	0652
0558	0558
0476	0476
0394	0394
0318	0318
0257	0257
0195	0195
0137	0137
0096	0096
0055	0055
0031	0031
0010	0010
0000	0000
0000	0000

```
0000                                                    0000
0006                                                    0006
0027                                                    0027
0047                                                    0047
0088                                                    0088
0129                                                    0129
0183                                                    0183
0245                                                    0245
0306                                                    0306
0378                                                    0378
0460                                                    0460
0542                                                    0542
0632                                                    0632
0722                                                    0722
0821                                                    0821
0929                                                    0929
1042                                                    1042
1158                                                    1158
1278                                                    1278
1388                                                    1388
1511                                                    1511
1650                                                    1650
1772                                                    1772
1895                                                    1895
$finish called from file "tb.v", line 40.                <
$finish at simulation time          1608                <
        V C S   S i m u l a t i o n   R e p o r t       <
Time: 1608                                              <
CPU Time:      0.170 seconds;       Data structure size:   0. <
Mon Nov 24 18:27:44 2025                                <
[dthornburg@linux402604 samples_100]$
```

Part 3 80 Samples:

Diff (should output no numbers if all match):

```
[dthornburg@linux402604 samples_80]$ diff  <(./simv -no_save | sed '1,3d') <(./main)
321,326d320
< $finish called from file "tb.v", line 40.
< $finish at simulation time                 1288
<           V C S   S i m u l a t i o n   R e p o r t
< Time: 1288
< CPU Time:      0.180 seconds;       Data structure size:   0.0Mb
< Mon Nov 24 18:41:57 2025
[dthornburg@linux402604 samples_80]$
```

Sdiff (output all numbers when matching):

```
[dthornburg@linux402604 samples_80]$ sdiff  <(./simv -no_save | sed '1,3d') <(./main)
2032                                                            2032

2201                                                            2201

2355                                                            2355

2521                                                            2521

2678                                                            2678

2815                                                            2815

2969                                                            2969

3106                                                            3106

3247                                                            3247

3369                                                            3369

3487                                                            3487

3590                                                            3590

3692                                                            3692

3782                                                            3782

3858                                                            3858
```

3935	3935
3987	3987
4035	4035
4060	4060
4080	4080
4080	4080
4080	4080
4060	4060
4035	4035
3987	3987
3936	3936
3859	3859
3782	3782
3693	3693
3591	3591
3488	3488
3370	3370
3249	3249
3108	3108
2970	2970
2817	2817
2679	2679
2523	2523
2356	2356
2203	2203
2033	2033
1879	1879

1726	1726
1560	1560
1403	1403
1265	1265
1112	1112
0974	0974
0833	0833
0711	0711
0593	0593
0490	0490
0388	0388
0298	0298
0221	0221
0145	0145
0093	0093
0045	0045
0019	0019
0000	0000
0000	0000
0000	0000
0018	0018
0044	0044
0092	0092
0143	0143
0219	0219
0296	0296
0385	0385
0487	0487

0590	0590
0708	0708
0829	0829
0970	0970
1107	1107
1261	1261
1398	1398
1555	1555
1721	1721
1875	1875
2029	2029
2198	2198
2352	2352
2517	2517
2675	2675
2813	2813
2966	2966
3103	3103
3245	3245
3367	3367
3485	3485
3588	3588
3690	3690
3780	3780
3857	3857
3934	3934
3986	3986

4034	4034
4060	4060
4080	4080
4080	4080
4080	4080
4061	4061
4035	4035
3988	3988
3937	3937
3861	3861
3784	3784
3695	3695
3593	3593
3490	3490
3372	3372
3251	3251
3111	3111
2973	2973
2820	2820
2682	2682
2526	2526
2359	2359
2206	2206
2037	2037
1882	1882
1729	1729
1563	1563
1406	1406

1267	1267
1115	1115
0977	0977
0836	0836
0713	0713
0596	0596
0492	0492
0390	0390
0300	0300
0223	0223
0146	0146
0094	0094
0045	0045
0020	0020
0000	0000
0000	0000
0000	0000
0018	0018
0043	0043
0091	0091
0142	0142
0218	0218
0294	0294
0383	0383
0485	0485
0588	0588
0706	0706

```
0827                                                    0827

0967                                                    0967

1104                                                    1104

1258                                                    1258

1395                                                    1395

1551                                                    1551

1718                                                    1718

1872                                                    1872

$finish called from file "tb.v", line 40.                    <
$finish at simulation time              1288                 <
        V C S   S i m u l a t i o n   R e p o r t            <
Time: 1288                                                        <
CPU Time:      0.180 seconds;       Data structure size:   0. <
Mon Nov 24 18:39:45 2025                                          <
[dthornburg@linux402604 samples_80]$ ^C
[dthornburg@linux402604 samples_80]$
```

Conclusion

Both sine-wave generator architectures were successfully implemented, verified, and shown to meet the specifications outlined in the Lab 7 handout.

For Parts 1 and 2, both the 8 to 12-bit interpolated designs using two ROM tables produced output sequences that matched the C model exactly across all tested waveform densities (100 and 75 samples per cycle). This was confirmed objectively using both diff and sdiff comparisons, which returned no mismatches after the waveform files were aligned in length.

In Part 3, the architecture was modified to use a single ROM. The design now outputs a new 12-bit sample every other clock cycle, and the Sample signal asserts when a new output is valid. The phase accumulator update rate was also reduced to every other cycle to match the single-ROM timing constraints. After these adjustments, the resulting hardware outputs again matched the C model. The 100 sample variant also matched part 1, which was expected.

Across all parts, the waveform generator consistently produced the correct 12-bit samples specified by the interpolated sine-wave algorithm. The matches between the hardware and C model outputs demonstrate that the Verilog implementations are functionally correct and that the datapath, control logic, interpolation behavior, and timing requirements are all implemented properly.

Appendix

C Code (part 1)

```
#include <stdio.h>
#include <stdlib.h>
#include "parameters.h"
#include "sintable.h"
#include <math.h>

// ALL "* CYCLES" were added in

int outputWave[SAMPLES_PER_CYCLE * CYCLES];
FILE *outfile;

int phase;
double ideal;



/* run this program using the console pauser or add your own getch, system("pause") or input
loop */

int main(int argc, char *argv[]) {
  //        printf("starting\n");
            fill_wavetable();
            clearPhaseAcc();
            setPhaseReg(PHASE_INC);
//          printf("%d = PHASE_INC\n", PHASE_INC);
//          printf("%d = PHASE_FRAC_BITS\n", PHASE_FRAC_BITS);
//          printf("%d = PHASE_FRAC_MASK\n", PHASE_FRAC_MASK);
            outfile = fopen("outputwave.csv","w");

/* generate 1 cycle worth of data and print it out */
            phase = getPhase();
        int s1,s2;
            int frac;
            for (int i = 0; i < SAMPLES_PER_CYCLE * CYCLES; i++) {
                        s1 = table_lookup(phase);
                        s2 = table_lookup((phase + PHASE_UNIT));
                        frac = phase & PHASE_FRAC_MASK;
//                      printf("%d, %d, %d, %d\n", s1, s2, frac, PHASE_UNIT);
                        outputWave[i] = (((frac * s2) + ((PHASE_UNIT-frac) * s1)) >>
(WAVETABLE_OUTPUT_BITS + PHASE_FRAC_BITS - INTERP_OUTPUT_BITS)) &
INTERP_OUTPUT_MASK;
                        //          fprintf(outfile, "%d, %d\n",phase >> (PHASE_FRAC_BITS),
outputWave[i]);
                        phase = incPhase();
            }
            if (phase < PHASE_INC) phase += (WAVETABLE_SIZE<< PHASE_FRAC_BITS);
            double freqErr = (double) phase / (double) WAVETABLE_SIZE / (double) (1 <<
PHASE_FRAC_BITS) - 1.0;
//          printf ("Phase = %f, Range = %f\n",(double) phase, (double) WAVETABLE_SIZE);
```

```
//          printf ("Frequency Error = %.3f %%\n", 100.0*freqErr);
//          do an error calculation
            double error = 0;
            for (int i = 0; i < SAMPLES_PER_CYCLE * CYCLES; i++) {
              /* added code */
              printf("%04d\n", (int) outputWave[i],(int) i);
              printf("\n");
              /* added code */
                        ideal = sin((2.0 * M_PI * (double) (i) / (double) WAVETABLE_SIZE)  + 1.0) * 0.5 *
SIN_SCALE * WAVETABLE_RANGE;
                        error += ((double) outputWave[i] - ideal) * ((double) outputWave[i] - ideal) ;
            }
            //          printf ("SNR = %f\n", WAVETABLE_RANGE/ (error/ (double)
(SAMPLES_PER_CYCLE * CYCLES)));

            fclose(outfile);
            return 0;
}
```

```
//sintable.c
#include "parameters.h"
#include <math.h>
#include <stdio.h>
#include <stdlib.h>

int wavetable[WAVETABLE_SIZE];
int phaseAcc;
int phaseReg;

void fill_wavetable(){
            double sample, phase;
            FILE *f;
            f=fopen("wavetable.mem", "w");
            for (int i = 0; i < WAVETABLE_SIZE; i++) {
                        phase = (2.0 * M_PI * (double) (i) / (double) WAVETABLE_SIZE);
                        sample = (sin(phase) + 1.0) * 0.5 * SIN_SCALE * WAVETABLE_RANGE;
                        wavetable[i] = ((int) sample) & WAVETABLE_OUTPUT_MASK;
                        //          printf("i = %d, phase = %f, sample = %f, wavetable[%d], = %d\n",i, phase,
sample, i, wavetable[i]);
                        fprintf(f,"%x\n", wavetable[i]);
              }
            fclose(f);
}

int table_lookup(int phase) {
            return wavetable[(phase & PHASE_MASK) >> PHASE_FRAC_BITS] &
WAVETABLE_OUTPUT_MASK;
}

void clearPhaseAcc() {
            phaseAcc = 0;
}

int incPhase() {
```

```
	phaseAcc = (phaseAcc + phaseReg) & PHASE_MASK;
	return phaseAcc;
}

int getPhase() {
	return phaseAcc >> PHASE_FRAC_BITS;
}

void setPhaseReg(int phaseinc) {
	phaseReg = phaseinc & PHASE_MASK;
//	printf("Phase Increment is %d\n",phaseinc);
}
```

```
//sintable.h
void fill_wavetable();
int table_lookup(int phase);
void clearPhaseAcc();
int incPhase();
int getPhase();
void setPhaseReg(int phaseinc);
```

```
//parameters.h
#define WAVETABLE_ADDRESS_SIZE 7
#define WAVETABLE_SIZE (1 << WAVETABLE_ADDRESS_SIZE)
#define WAVETABLE_OUTPUT_BITS 8		// must not exceed size of int
#define WAVETABLE_OUTPUT_MASK ((1 << WAVETABLE_OUTPUT_BITS) - 1)
#define WAVETABLE_RANGE (double)(1 << WAVETABLE_OUTPUT_BITS)
#define PHASE_TOTAL_BITS 16
#define PHASE_INTEGER_BITS WAVETABLE_ADDRESS_SIZE
#define PHASE_FRAC_BITS (PHASE_TOTAL_BITS - PHASE_INTEGER_BITS)
#define PHASE_MASK ((1 << PHASE_TOTAL_BITS) - 1)
#define PHASE_UNIT (1 << PHASE_FRAC_BITS)
#define PHASE_FRAC_MASK ((1 << PHASE_FRAC_BITS)-1)
#define INTERP_OUTPUT_BITS 12
#define INTERP_OUTPUT_MASK ((1 << INTERP_OUTPUT_BITS) - 1)
#define SIN_SCALE 0.99999
/* added */
#define CYCLES 2
/* added */
#define SAMPLES_PER_CYCLE 100
#define SIGNAL_LENGTH 128
#define PHASE_INC ((WAVETABLE_SIZE * (1 << PHASE_FRAC_BITS))/SAMPLES_PER_CYCLE)
```

VERILOG (part 1)

```
module adder(
	input [15:0] phase_reg_out,
	input [15:0] phase_accum_out,
	output [15:0] adder_out //maybe 16:0
);
	assign adder_out = phase_reg_out + phase_accum_out;
```

```
endmodule
```

```
module phase_register(
	input [15:0] phase_in, //7.9
	input load,
	input clk,
	output reg [15:0] phase_reg_out
);
	always@(posedge clk) if(load) phase_reg_out <= phase_in & ((1 << 5'd16) - 1);
endmodule
```

```
module phase_accum(
	input clk,
	input rst,
	input [15:0] adder_out, //adder_out
	output reg [15:0] phase_accum_out
);
	always@(posedge clk) begin
		if(rst) phase_accum_out <= 0;
		else phase_accum_out <=  adder_out & ((1 << 5'd16) - 1);
	end
endmodule
```

```
module rounder (
	input [15:0] phase_accum_out,
	output [8:0] interpolator_in, //decimals
	output [6:0] rounder_out //integers
);

	assign rounder_out =  phase_accum_out[15:9];
	assign interpolator_in =  phase_accum_out[8:0];

endmodule
```

```
module adder2(
	input [6:0] rounder_out,
	output [6:0] adder_out
);

	assign adder_out = rounder_out + 1'b1;

endmodule
```

```
module wave_table (
	input [6:0] wave_in,
	output reg [7:0] wave_out
);
	reg [7:0] mem [0:127];
	initial $readmemh("wavetable.mem", mem);
	always@(*) wave_out = mem[wave_in];
```

```
endmodule
```

```
module interpolator (
	input [8:0] interpolator_in, //decimals
	input [7:0] wave_table1_out,
	input [7:0] wave_table2_out,
	output [11:0] interpolator_out
);
	assign interpolator_out = (((interpolator_in * wave_table2_out) + (((1 << 
4'd9)-interpolator_in) * wave_table1_out)) >> (5'd8+5'd9-5'd12)) & ((1 << 5'd12)-1);
endmodule
```

```
module sin_wave_gen(
	input [15:0] phase_in,
	input load,
	input clk,
	input rst,
	output [11:0] out
);

	wire [15:0] pr_one_out;
	phase_register pr_one(
		.clk(clk),
		.load(load),
		.phase_in(phase_in),
		.phase_reg_out(pr_one_out)
	);



	wire [15:0] adder_one_out;
	wire [15:0] pa_one_out;
	adder adder_one(
		.phase_reg_out(pr_one_out),
		.phase_accum_out(pa_one_out),
		.adder_out(adder_one_out)
	);
	phase_accum pa_one(
		.clk(clk),
		.rst(rst),
		.adder_out(adder_one_out),
		.phase_accum_out(pa_one_out)
	);



	wire [8:0] interpolator_in;
	wire [6:0] rounder_out;
	rounder r_one(
		.phase_accum_out(pa_one_out),
		.interpolator_in(interpolator_in),
		.rounder_out(rounder_out)
	);
```

```
	wire [6:0] adder_two_out;
	adder2 adder_two(
		.rounder_out(rounder_out),
		.adder_out(adder_two_out)
	);

	wire [7:0] wave_one_out;
	wave_table wave_one(
		.wave_in(rounder_out),
		.wave_out(wave_one_out)
	);

	wire [7:0] wave_two_out;
	wave_table wave_two(
		.wave_in(adder_two_out),
		.wave_out(wave_two_out)
	);



	interpolator interpolator_one(
		.interpolator_in(interpolator_in),
		.wave_table1_out(wave_one_out),
		.wave_table2_out(wave_two_out),
		.interpolator_out(out)
	);



endmodule
```

```
module testbench();

reg [15:0] phase_in;
reg load, clk, rst;
wire [11:0] out;
reg [7:0] i;
sin_wave_gen dut(
	.phase_in(phase_in),
	.load(load),
	.clk(clk),
	.rst(rst),
	.out(out)
);

initial begin
	//$display("Start of Sin Wave Test \n");
	clk=0; rst=1; load=1; phase_in=(8'd128 * 10'd512) / 100;
	#2;
	clk=1; rst=1; load=1; phase_in=(8'd128 * 10'd512) / 100;
	#2;
```

```
        $display("%04d\n", out);
        #2;
        clk=0; rst=0; load=0; phase_in=0;
        #2;
        clk=1; rst=0; load=0; phase_in=0;
        #2;
        $display("%04d\n", out);
        #2;
        for (i=0; i < 198; i= i + 1) begin //198 b/c we already did two cycles
                clk=0;
                #2;
                clk=1;
                #2;
                $display("%04d\n", out);
                #2;
        end
        clk=0; rst=1; load=0;
        #2;
        clk=1; rst=1; load=0;
        #2;
        clk=0; rst=0; load=0;
        #2;
        clk=1; rst=0; load=0;
        #2;
        //$display("End of Sin Wave Test");
$finish;

end

endmodule
```

Wavetable.mem:

```
7f
86
8c
92
98
9f
a5
ab
b0
b6
bc
c1
c7
cc
d1
d5
da
de
e2
e6
ea
ed
```

f0
f3
f6
f8
fa
fc
fd
fe
ff
ff
ff
ff
ff
fe
fd
fc
fa
f8
f6
f3
f0
ed
ea
e6
e2
de
da
d5
d1
cc
c7
c1
bc
b6
b0
ab
a5
9f
98
92
8c
86
7f
79
73
6d
67
60
5a
54
4f
49
43
3e
38

```
33
2e
2a
25
21
1d
19
15
12
f
c
9
7
5
3
2
1
0
0
0
0
0
1
2
3
5
7
9
c
f
12
15
19
1d
21
25
2a
2e
33
38
3e
43
49
4f
54
5a
60
67
6d
73
79
```

C Code (part 2)

```
//parameters.h
#define WAVETABLE_ADDRESS_SIZE 7
#define WAVETABLE_SIZE (1 << WAVETABLE_ADDRESS_SIZE)
#define WAVETABLE_OUTPUT_BITS 8          // must not exceed size of int
#define WAVETABLE_OUTPUT_MASK ((1 << WAVETABLE_OUTPUT_BITS) - 1)
#define WAVETABLE_RANGE (double)(1 << WAVETABLE_OUTPUT_BITS)
#define PHASE_TOTAL_BITS 16
#define PHASE_INTEGER_BITS WAVETABLE_ADDRESS_SIZE
#define PHASE_FRAC_BITS (PHASE_TOTAL_BITS - PHASE_INTEGER_BITS)
#define PHASE_MASK ((1 << PHASE_TOTAL_BITS) - 1)
#define PHASE_UNIT (1 << PHASE_FRAC_BITS)
#define PHASE_FRAC_MASK ((1 << PHASE_FRAC_BITS)-1)
#define INTERP_OUTPUT_BITS 12
#define INTERP_OUTPUT_MASK ((1 << INTERP_OUTPUT_BITS) - 1)
#define SIN_SCALE 0.99999
/* added */
#define CYCLES 2
/* added */
#define SAMPLES_PER_CYCLE 75
#define SIGNAL_LENGTH 128
#define PHASE_INC ((WAVETABLE_SIZE * (1 << PHASE_FRAC_BITS))/SAMPLES_PER_CYCLE)
```

VERILOG (part 2)

```
module testbench();

reg [15:0] phase_in;
reg load, clk, rst;
wire [11:0] out;
reg [7:0] i;
sin_wave_gen dut(
        .phase_in(phase_in),
        .load(load),
        .clk(clk),
        .rst(rst),
        .out(out)
);

initial begin
        //$display("Start of Sin Wave Test \n");
        clk=0; rst=1; load=1; phase_in=(8'd128 * 10'd512) / 75;
        #2;
        clk=1; rst=1; load=1; phase_in=(8'd128 * 10'd512) / 75;
        #2;
        $display("%04d\n", out);
        #2;
        clk=0; rst=0; load=0; phase_in=0;
        #2;
        clk=1; rst=0; load=0; phase_in=0;
        #2;
        $display("%04d\n", out);
        #2;
        for (i=0; i < 148; i= i + 1) begin //148 b/c we already did two cycles
                clk=0;
```

```
                #2;
                clk=1;
                #2;
                $display("%04d\n", out);
                #2;
        end
        clk=0; rst=1; load=0;
        #2;
        clk=1; rst=1; load=0;
        #2;
        clk=0; rst=0; load=0;
        #2;
        clk=1; rst=0; load=0;
        #2;
        //$display("End of Sin Wave Test");
$finish;

end

endmodule
```

All C Code is the same as part 1 100 Samples for Part 3 100 Samples

VERILOG (part 3 | 100 Samples) only changed code added

```
module phase_accum(
        input clk,
        input [15:0] adder_out, //adder_out
        output reg [15:0] phase_accum_out
);
        reg count;
        initial begin
                count<=0;
                phase_accum_out <= 16'd0;
        end
        always@(posedge clk) begin
                if(!count) count<=1;
                else begin
                        phase_accum_out <=  adder_out & ((1 << 5'd16) - 1);
                        count<=0;
                end
        end
        //initial phase_accum_out <= 16'd0;
        //always@(posedge clk)phase_accum_out <=  adder_out & ((1 << 5'd16) - 1);
endmodule
```

```
module control_path(
        input rst,
```

```
	output reg sample,
	input clk,
	output reg ctrl_path_output,
	output reg interp_sel
);

	reg state;
	always@(posedge clk) begin
		if (rst) begin
			state <=0;
			ctrl_path_output <=0;
			sample <=1;
			interp_sel <=0;
		end
		else begin
			if(!state) begin
				ctrl_path_output=0;
				interp_sel=0;
				sample=0;
				state=1;
			end
			else begin
				ctrl_path_output=1;
				interp_sel=1;
				sample=1;
				state=0;
			end
		end
	end
endmodule
```

```
module sin_wave_gen(
	input [15:0] phase_in,
	input load,
	input clk,
	input rst,
	output sample,
	output [11:0] out
);

	wire [15:0] pr_one_out;
	phase_register pr_one(
		.clk(clk),
		.load(load),
		.phase_in(phase_in),
		.phase_reg_out(pr_one_out)
	);



	wire [15:0] adder_one_out;
	wire [15:0] pa_one_out;
	adder adder_one(
		.phase_reg_out(pr_one_out),
```

```
		.phase_accum_out(pa_one_out),
		.adder_out(adder_one_out)
	);
	phase_accum pa_one(
		.clk(clk),
		.adder_out(adder_one_out),
		.phase_accum_out(pa_one_out)
	);



	wire [8:0] interpolator_in;
	wire [6:0] rounder_out;
	rounder r_one(
		.phase_accum_out(pa_one_out),
		.interpolator_in(interpolator_in),
		.rounder_out(rounder_out)
	);

	wire ctrl_path_out, interp_sel;
	control_path ctrl_one(
		.rst(rst),
		.sample(sample),
		.clk(clk),
		.ctrl_path_output(ctrl_path_out),
		.interp_sel(interp_sel)
	);

	wire [6:0] adder_two_out;
	adder2 adder_two(
		.rounder_out(rounder_out),
		.ctrl_path_out(ctrl_path_out),
		.adder_out(adder_two_out)
	);


	wire [7:0] wave_two_out;
	wave_table wave_two(
		.wave_in(adder_two_out),
		.wave_out(wave_two_out)
	);



	//Logic for interpolator must now be handled outside as there's no clk in the interpolator
	reg [7:0] table1;
	interpolator interpolator_one(
		.interpolator_in (interpolator_in),
		.wave_table1_out (table1),
		.wave_table2_out (wave_two_out),
		.interpolator_out(out)
	);

	always @(posedge clk) begin
		if (rst) table1 <= 7'h7f; //first val in mem table
```

```
                    else if (!interp_sel) table1 <= wave_two_out;
          end

endmodule
```

```
module testbench();

reg [15:0] phase_in;
reg load, clk, rst;
wire sample;
wire [11:0] out;
reg [9:0] i;
sin_wave_gen dut(
          .phase_in(phase_in),
          .load(load),
          .clk(clk),
          .rst(rst),
          .out(out),
          .sample(sample)
);

initial begin
          //$display("Start of Sin Wave Test \n");
          clk=0; rst=1; load=1; phase_in=(8'd128 * 10'd512) / 100;
          #2;
          clk=1; rst=1; load=1; phase_in=(8'd128 * 10'd512) / 100; // reset state
          #2;
          if (sample) $display("%04d\n", out);
          for (i=0; i < 399; i= i + 1) begin //399 b/c we already did one cycle and it needs to be
double what it was (200*2-1=399)
                    clk=0; rst=0; load=0; phase_in=0;
                    #2;
                    clk=1; rst=0; load=0; phase_in=0;
                    #2;
                    if (sample) $display("%04d\n", out);
          end
          clk=0; rst=1; load=0;
          #2;
          clk=1; rst=1; load=0;
          #2;
          clk=0; rst=0; load=0;
          #2;
          clk=1; rst=0; load=0;
          #2;
          //$display("End of Sin Wave Test");
$finish;

end

endmodule
```

C Code (part 3 | 80 samples)

```
//parameter.h
#define WAVETABLE_ADDRESS_SIZE 7
#define WAVETABLE_SIZE (1 << WAVETABLE_ADDRESS_SIZE)
#define WAVETABLE_OUTPUT_BITS 8          // must not exceed size of int
#define WAVETABLE_OUTPUT_MASK ((1 << WAVETABLE_OUTPUT_BITS) - 1)
#define WAVETABLE_RANGE (double)(1 << WAVETABLE_OUTPUT_BITS)
#define PHASE_TOTAL_BITS 16
#define PHASE_INTEGER_BITS WAVETABLE_ADDRESS_SIZE
#define PHASE_FRAC_BITS (PHASE_TOTAL_BITS - PHASE_INTEGER_BITS)
#define PHASE_MASK ((1 << PHASE_TOTAL_BITS) - 1)
#define PHASE_UNIT (1 << PHASE_FRAC_BITS)
#define PHASE_FRAC_MASK ((1 << PHASE_FRAC_BITS)-1)
#define INTERP_OUTPUT_BITS 12
#define INTERP_OUTPUT_MASK ((1 << INTERP_OUTPUT_BITS) - 1)
#define SIN_SCALE 0.99999
/* added */
#define CYCLES 2
/* added */
#define SAMPLES_PER_CYCLE 80
#define SIGNAL_LENGTH 128
#define PHASE_INC ((WAVETABLE_SIZE * (1 << PHASE_FRAC_BITS))/SAMPLES_PER_CYCLE)
```

VERILOG (part 3 | 80 samples)

```
module testbench();

reg [15:0] phase_in;
reg load, clk, rst;
wire sample;
wire [11:0] out;
reg [9:0] i;
sin_wave_gen dut(
        .phase_in(phase_in),
        .load(load),
        .clk(clk),
        .rst(rst),
        .out(out),
        .sample(sample)
);

initial begin
        //$display("Start of Sin Wave Test \n");
        clk=0; rst=1; load=1; phase_in=(8'd128 * 10'd512) / 80;
        #2;
        clk=1; rst=1; load=1; phase_in=(8'd128 * 10'd512) / 80; // reset state
        #2;
        if (sample) $display("%04d\n", out);
        for (i=0; i < 319; i= i + 1) begin //319 b/c we already did one cycle and it needs to be
double what it was (160*2-1=319)
```

```
                clk=0; rst=0; load=0; phase_in=0;
                #2;
                clk=1; rst=0; load=0; phase_in=0;
                #2;
                if (sample) $display("%04d\n", out);
        end
        clk=0; rst=1; load=0;
        #2;
        clk=1; rst=1; load=0;
        #2;
        clk=0; rst=0; load=0;
        #2;
        clk=1; rst=0; load=0;
        #2;
        //$display("End of Sin Wave Test");
$finish;

end

endmodule
```